Working With Dictionaries

In previous lessons, we learned that dictionaries store information as key–value pairs, and we practiced **accessing**, **adding**, **updating**, **removing**, and **checking entries**. Python also provides dictionary methods that make common tasks easier.

This lesson introduces get(), keys(), values(), items(), and update() methods.

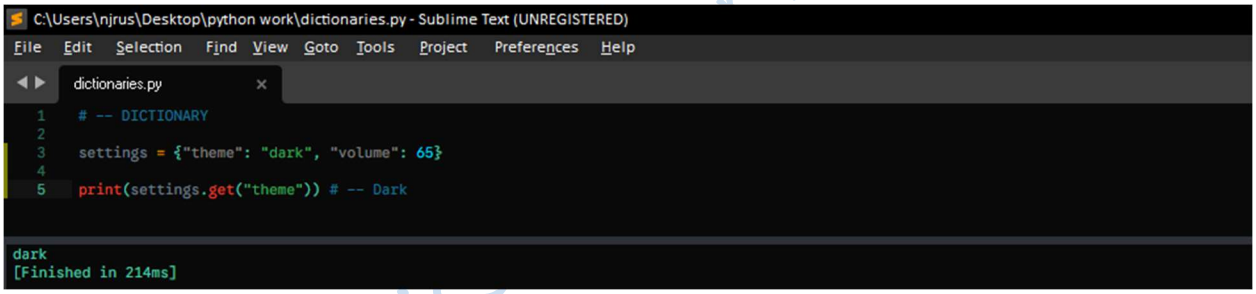
1. get() Method

The get() method is used to return the value for a key. They syntax for this method is as follows:

```
dictionary_name.get(key, default_value)
```

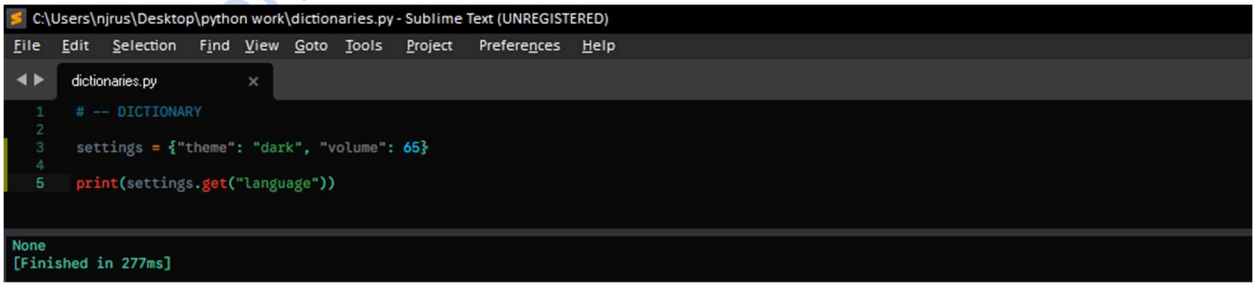
If the key you are trying to retrieve is missing, it returns None by default or the value you supply at "default_value" . Note that this method does not add that key or raise KeyError for a missing key. See some get() method examples below:

Retrieve an existing value

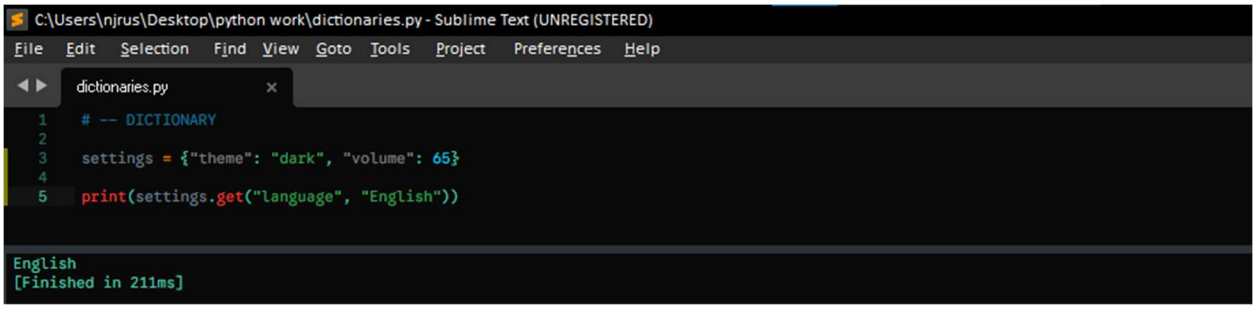


The "theme" key exists, so its value is returned.

Supply a default



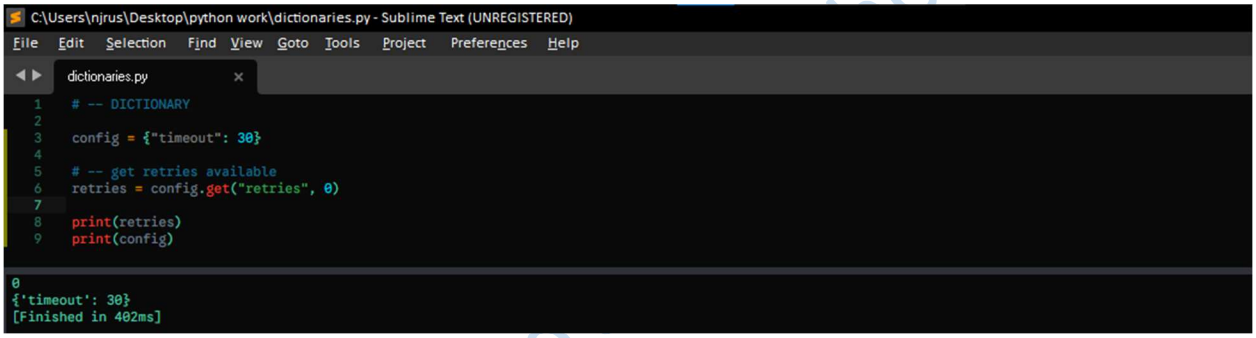
Since "language" is missing, and no value was supplied as fallback, None is returned instead. If we supplied a fallback value,



the supplied fallback is returned.

Default without insertion

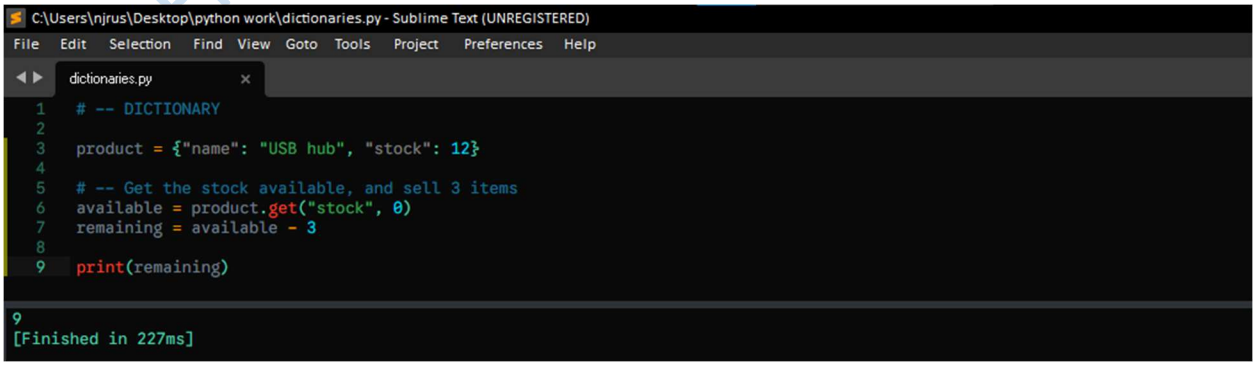
Retrieving a key's value with the get() method does not create a new dictionary.



Here, we try to get the number of retries available for a certain user in, say, a website. Since no retries are available in his data, the default value is returned and is assigned to retries. Note that printing the config data gives us the original dictionary. That is, the fallback 0 is returned, but the dictionary remains unchanged.

Use the result in a calculation

The returned values from get() method can also be used in your programs.

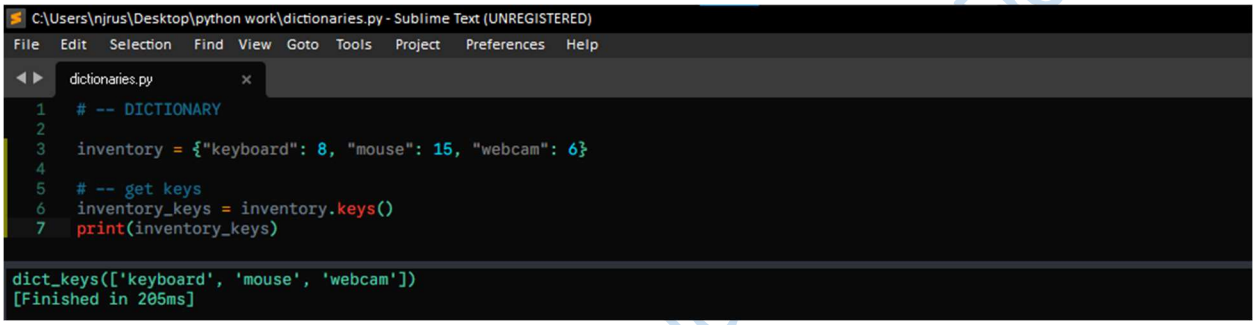


Here, we get the available stock for a particular product. A buyer then purchases three of those products and we get the remaining stock after the purchase.

2. keys() Method

The keys() method is used to return a dictionary view containing all keys. It is useful for inspecting or looping through dictionary keys.

View keys



The view displays the dictionary's keys.

Looping through keys



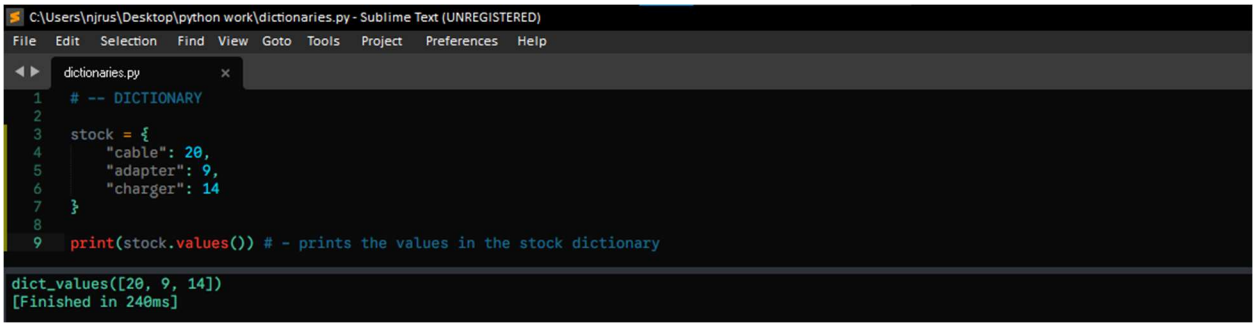
Each loop iteration assigns one key to item.

3. values()

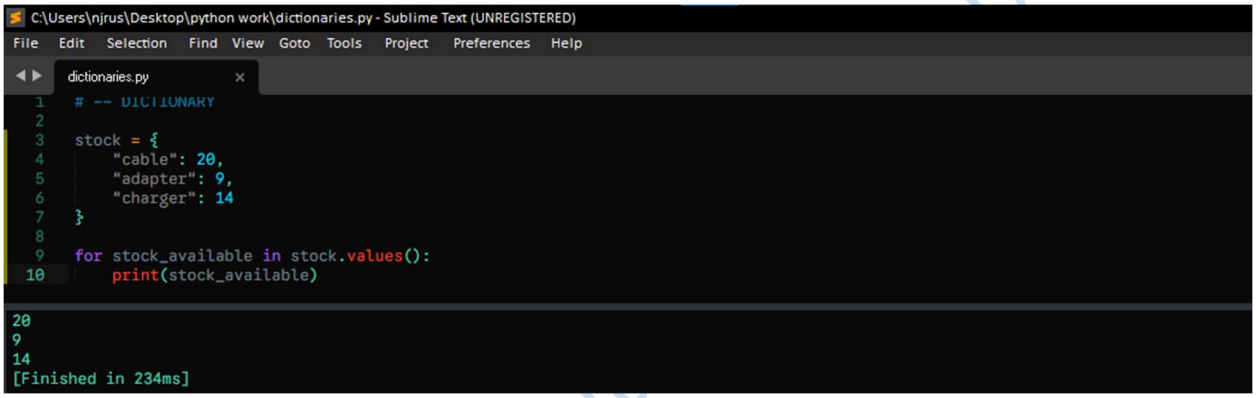
The values() method, on the other hand, returns a view of all values in dictionary iteration order. Values may repeat, because dictionary keys are unique but values do not have to be.

View values

This only displays the values currently available in the dictionary.



Looping through values

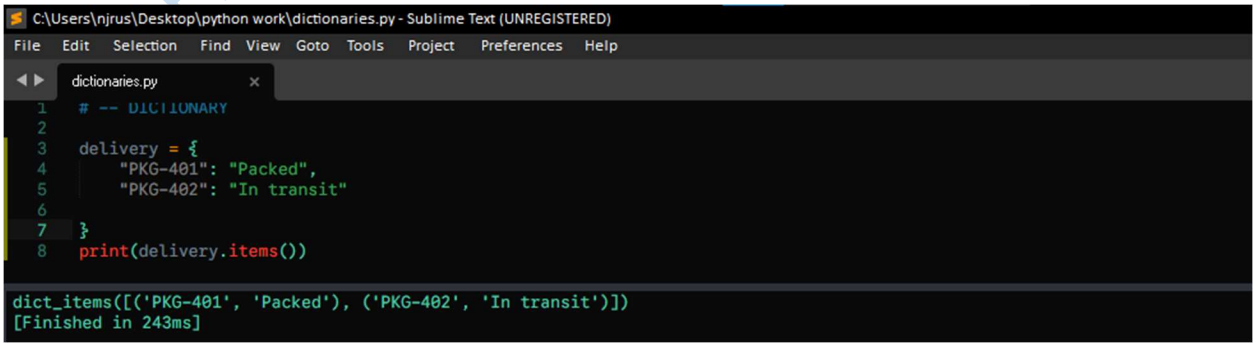


Every value in the stock dictionary gets printed.

4. items()

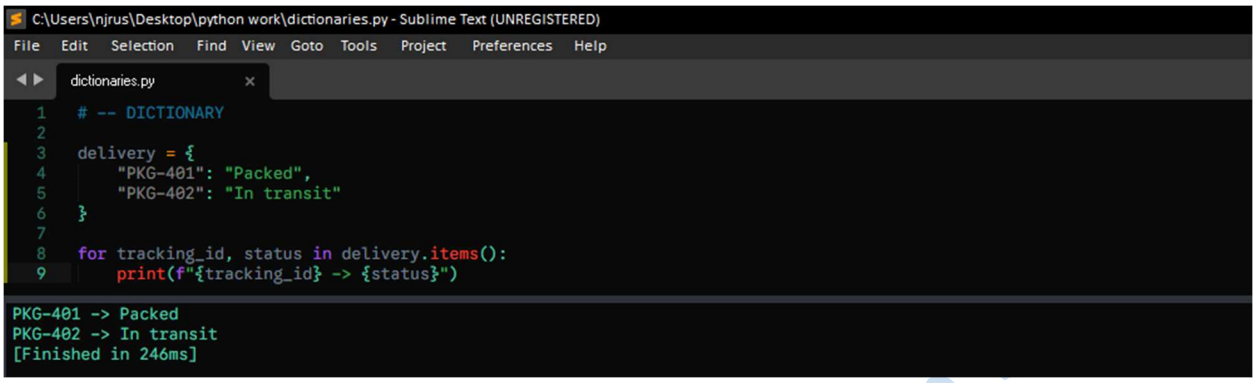
The items() method returns a view of key–value pairs. It is especially useful when you need both the key and its value in a loop.

View pairs



Each entry is shown as a tuple of key and value. (**Tuples shall be discussed later**)

Loop through pairs



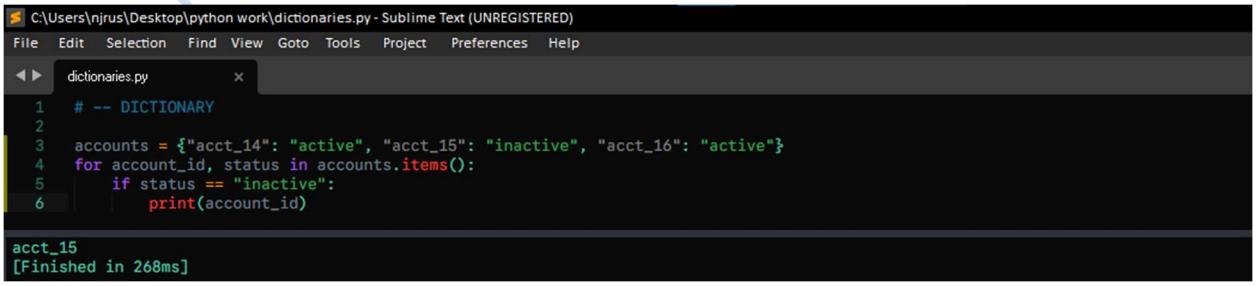
The two loop variables receive the key and matching value.

Display product prices



Each product is printed with its price.

Make a decision from values

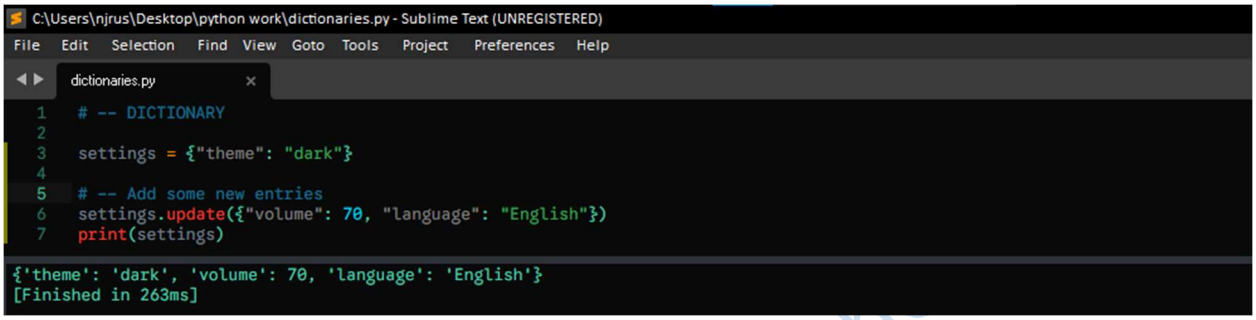


Print only accounts whose status is inactive.

5. update()

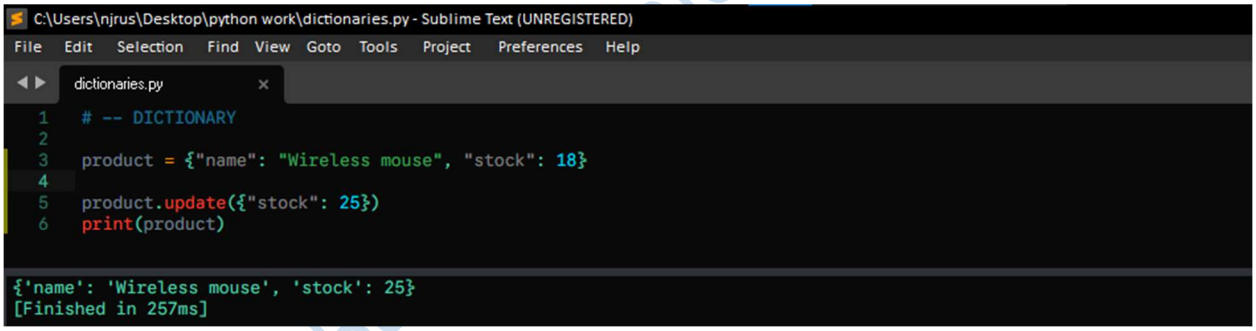
The update() method adds entries from another mapping or iterable of key–value pairs. New keys are added; matching existing keys have their values replaced. It modifies the original dictionary and returns None.

Add several entries



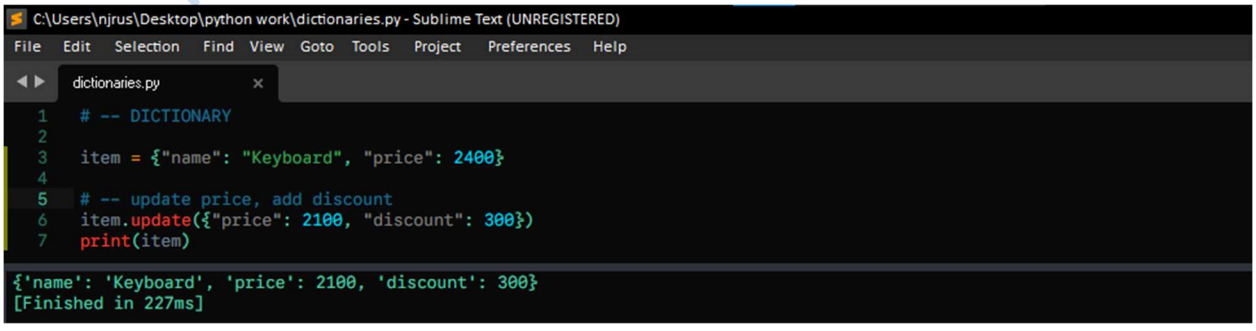
Both new keys are added to settings.

Update a value



The existing "stock" value is replaced; no duplicate key is created.

Add and update together



The price changes and a new discount entry is added.

Summary

Method	Purpose	Changes dictionary?
get(key, default)	Retrieve a value or fallback	No
keys()	View keys	No
values()	View values	No
items()	View key–value pairs	No
update(...)	Add or replace entries	Yes

From Basics to Junior Python Dev